26-2 유니티 스터디

3 주차

정혜원

오늘 만드는 것 & 진행 순서

물리 이벤트로 같은 레벨의 Dongle 합치기

① 충돌 조건과 합칠 위치 결정
② 코루틴으로 이동하며 숨기기 / Animator로 성장하기
③ 성장한 최대 레벨을 다음 Dongle 생성에 반영하기

코드 슬라이드 읽는 방법

작성 위치, 코드, 설명을 같은 화면에서 확인해요

초록 안내: 어느 파일의 어떤 위치에 작성하는지
회색 코드: 이미 작성한 부분
색이 있는 코드: 이번에 추가하거나 수정하는 부분
오른쪽 설명: 코드의 역할과 주의할 점

물리 이벤트와 상대방 확인

Dongle.cs / Drop() 아래에 새 함수 추가 / 클래스의 마지막 } 위

```
void OnCollisionStay2D(Collision2D collision)
{
    if (collision.gameObject.tag == "Dongle")
    {
        Dongle other =
            collision.gameObject.GetComponent<Dongle>();
    }
}
```

충돌 중에 실행

OnCollisionStay2D()는 충돌 중인
상대의 정보를 collision으로 받아요.

Dongle끼리만 처리

상대의 태그가 "Dongle"일 때만
중괄호 안의 코드를 실행해요.

상대의 정보 가져오기

GetComponent<Dongle>()로
상대의 level과 isMerge를 읽어요.

합치기 조건 4개

Dongle.cs / isDrag 아래 변수 추가 + OnCollisionStay2D()의 other 선언 아래 if 추가

```
// 클래스 위쪽: isDrag 바로 아래
public bool isMerge;
// 아래: 태그 확인 if 안
Dongle other =
    collision.gameObject.GetComponent<Dongle>();
if (level == other.level && !isMerge
    && !other.isMerge && level < 7)
{
    // 다음 단계: 위치를 비교해요
}
```

① 같은 레벨

level == other.level
같은 레벨 두 개만 합쳐져요.

② 내 잠금 + ③ 상대 잠금

!isMerge && !other.isMerge
둘 다 합치는 중이 아니어야 해요.
3~4개가 닿아도 합치기는 1 대 1!

④ 최종 레벨 제한

level < 7: 레벨은 0~7이에요.
레벨 8이 없으므로 7은 합치지 않아요.

논리 연산자

! 는 부울 값을 반대로 바꿔요.
&& 는 모든 조건을 만족해야 통과해요.

내 위치와 상대방 위치

Dongle.cs / OnCollisionStay2D() / 네 가지 합치기 조건을 통과한 if 안

```
if (level == other.level && !isMerge
    && !other.isMerge && level < 7)
{
    float meX = transform.position.x;
    float meY = transform.position.y;
    float otherX = other.transform.position.x;
    float otherY = other.transform.position.y;
}
```

내 좌표

meX와 meY에 내 위치를 저장해요.
X는 좌우, Y는 높이예요.

상대 좌표

otherX와 otherY에는
상대 Dongle의 위치를 저장해요.

위치로 역할 결정

둘 중 누가 성장하고 누가 숨을지
다음 조건에서 비교할 준비예요.

성장할 Dongle 결정

Dongle.cs / OnCollisionStay2D() / otherY 선언 바로 아래

```
float otherY = other.transform.position.y;
if (meY < otherY
   || (meY == otherY && meX > otherX))
{
   other.Hide(transform.position);
   LevelUp();
}
```

내가 아래에 있을 때

meY < otherY이면 내가 성장해요.

같은 높이일 때

meY == otherY이면서 meX > otherX
오른쪽에 있는 Dongle이 성장해요.

|| 와 괄호

둘 중 하나만 만족해도 통과해요.
같은 높이 && 오른쪽은 ( )로 묶어요.
원화 키 + Shift로 | 를 두 번 입력!

숨기기와 성장 호출

상대는 내 위치로 이동해서 숨고,
나는 LevelUp()을 실행해요.
두 함수는 이어서 만들어요.

CircleCollider2D 준비

Dongle.cs / rigid 아래에 circle 선언 / Awake()에서 rigid 초기화 다음에 한 줄 추가

```
Rigidbody2D rigid;
CircleCollider2D circle;
Animator anim;
void Awake()
{
    rigid = GetComponent<Rigidbody2D>();
    circle = GetComponent<CircleCollider2D>();
    anim = GetComponent<Animator>();
}
```

변수 선언

CircleCollider2D circle;을
함수 밖의 변수 영역에 추가해요.

Awake()에서 초기화

GetComponent<CircleCollider2D>()로
내 콜라이더를 가져와 저장해요.

숨길 때 사용

흡수 중에는 circle.enabled로
Collider를 비활성화할 거예요.

숨기기 함수: Hide()

Dongle.cs / OnCollisionStay2D() 아래 / 다른 함수의 중괄호 밖에 새 함수 추가

```
public void Hide(Vector3 targetPos)
{
    isMerge = true;
    rigid.simulated = false;
    circle.enabled = false;
    StartCoroutine(HideRoutine(targetPos));
}
```

public과 목표 위치

상대방에서 호출하므로 public 사용.
other.Hide(transform.position)에서
넘긴 내 위치가 targetPos로 들어와요.

잠금과 물리 끄기

isMerge = true로 합치기 잠금.
simulated = false로 물리 끄기.
enabled = false로 Collider 끄기.

이동은 코루틴에서

목표 위치를 HideRoutine()에
한 번 더 넘겨 이동을 시작해요.

숨기기 코루틴: HideRoutine()

Dongle.cs / Hide() 바로 아래 / 전체 함수를 한 번에 작성

```
IEnumerator HideRoutine(Vector3 targetPos)
{
    int frameCount = 0;
    while (frameCount < 20)
    {
        frameCount++;
        transform.position = Vector3.Lerp(
            transform.position, targetPos, 0.5f);
        yield return null;
    }
    isMerge = false;
    gameObject.SetActive(false);
}
```

20프레임 동안 반복

IEnumerator로 코루틴을 선언해요.
frameCount++로 매번 1씩 증가.
빠뜨리면 반복이 끝나지 않아요.

Vector3.Lerp()

현재 위치, 목표 위치, 이동 강도 순서.
0.5f는 남은 거리의 절반만큼 이동.

한 프레임씩 쉬기

yield return null은 while 안에!
없으면 한 프레임에 전부 실행해요.

반복이 끝난 다음

while의 } 다음에 잠금을 풀고
SetActive(false)로 완전히 숨겨요.

숨기기 테스트

GameManager.cs / NextDongle() / lastDongle.level 줄을 테스트용으로 교체

```
void NextDongle()
{
    Dongle newDongle = GetDongle();
    lastDongle = newDongle;
    lastDongle.level = Random.Range(0, 2);
    lastDongle.gameObject.SetActive(true);
    StartCoroutine(WaitNext());
}
```

낮은 레벨만 생성

0~8 대신 0~2 범위로 임시 변경.
정수 최댓값은 제외하므로 0 또는 1.

저장 후 플레이

같은 레벨 두 개를 부딪혀 봐요.
위쪽 Dongle이 아래로 흡수되는지,
Hierarchy에서 비활성화되는지 확인.

같은 높이도 테스트

같은 높이에서는 오른쪽이 남아요.
성장 함수까지 만든 뒤 함께 확인!

성장 함수: LevelUp()

Dongle.cs / HideRoutine() 아래 / 함수 밖에 새 함수 추가

```
void LevelUp()
{
    isMerge = true;
    rigid.linearVelocity = Vector2.zero;
    rigid.angularVelocity = 0f;
    StartCoroutine(LevelUpRoutine());
}
```

성장 중에도 잠금

isMerge = true로 다른 Dongle이
합치기에 끼어들지 못하게 해요.

이동·회전 속도 초기화

움직이면 합치는 모양이 어색해요.
linearVelocity는 Vector2.zero,
angularVelocity는 0f로 초기화.

성장 코루틴 실행

물리를 전부 끌 필요는 없어요.
LevelUpRoutine()에서 성장 처리.
구버전은 linearVelocity 대신 velocity.

성장 코루틴: LevelUpRoutine()

Dongle.cs / LevelUp() 아래 / maxLevel 갱신은 뒤에서 추가

```
IEnumerator LevelUpRoutine()
{
    yield return new WaitForSeconds(0.2f);
    anim.SetInteger("Level", level + 1);
    yield return new WaitForSeconds(0.3f);
    level++;
    isMerge = false;
}
```

0.2초 대기

상대가 흡수되는 시간을 기다려요.
자기만 커지므로 상대 위치는 필요 X.

Animator 실행

SetInteger("Level", level + 1)로
다음 레벨의 성장 애니메이션 실행.

0.3초 뒤 실제 레벨 증가

크기와 Sprite가 바뀐 뒤 level++.
끝나면 isMerge = false로 잠금 해제.

레벨을 두 번 올리는 건가요?

level + 1은 Animator에 전달할 값.
level++만 실제 변수를 바꿔요.
시간차와 잠금으로 연속 합치기 방지.

성장 테스트

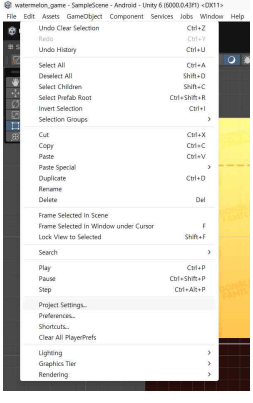
같은 레벨 두 개가 합쳐져 다음 레벨이 되는지 확인

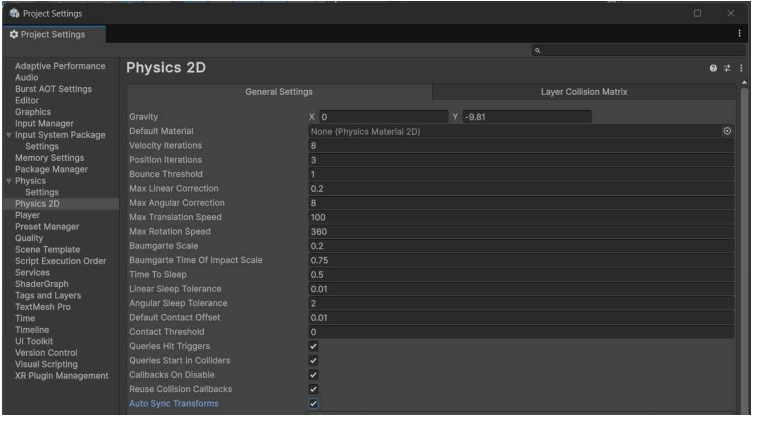
OnCollisionStay2D()에서 other.Hide()와 LevelUp()을 함께 호출
흡수 이동, Sprite 변경, 크기 증가가 순서대로 보이는지 플레이해 봐요

Physics 2D: Auto Sync Transforms

Edit / Project Settings / Physics 2D / Auto Sync Transforms 체크

① Edit에서 Project Settings 열기 ② Physics 2D에서 아래쪽 Auto Sync Transforms 체크





생성 레벨 범위: maxLevel

GameManager.cs / dongleGroup 아래 변수 선언 + NextDongle()의 랜덤 레벨 줄 교체

```
public Transform dongleGroup;
public int maxLevel;
// NextDongle() 안
lastDongle = newDongle;
lastDongle.level = Random.Range(0, maxLevel);
lastDongle.gameObject.SetActive(true);
```

생성 범위를 늘리는 이유

계속 0~1레벨만 나오면
높은 레벨까지 성장하기 오래 걸려요.

최대 레벨 저장

GameManager의 maxLevel에
지금까지 성장한 최대 레벨을 저장.

랜덤 범위에 사용

임시로 넣었던 숫자 2를
maxLevel로 교체해요.
최댓값은 생성 범위에 포함하지 않아요.

Dongle에 manager 참조 추가

Dongle.cs / 클래스 위쪽 / public int level; 바로 위

```
public class Dongle : MonoBehaviour
{
    public GameManager manager;
    public int level;
    public bool isDrag;
    public bool isMerge;
    // 아래: 기존 변수와 함수
}
```

GameManager가 필요해요

성장한 Dongle이 GameManager의 maxLevel을 수정하려면 참조가 필요.

함수 밖에 선언

public GameManager manager;를 클래스의 변수 영역에 추가해요.

생성할 때 연결

Dongle은 실행 중에 생성돼요. GameManager에서 만들면서 이 변수에 자신을 넣어줄 거예요.

생성할 때 manager 연결

GameManager.cs / NextDongle() / lastDongle = newDongle; 바로 다음

```
void NextDongle()
{
    Dongle newDongle = GetDongle();
    lastDongle = newDongle;
    lastDongle.manager = this;
    lastDongle.level = Random.Range(0, maxLevel);
    lastDongle.gameObject.SetActive(true);
    StartCoroutine(WaitNext());
}
```

manager 초기화

lastDongle.manager = this;를
레벨 설정과 활성화 전에 추가해요.

this의 의미

현재 NextDongle()을 실행하는
GameManager 스크립트 자신이에요.

연결 후 사용 가능

이제 Dongle에서 manager.maxLevel로
GameManager의 변수에 접근해요.

성장 후 최대 레벨 갱신

Dongle.cs / LevelUpRoutine() / level++; 다음, isMerge = false; 위

```
IEnumerator LevelUpRoutine()
{
    yield return new WaitForSeconds(0.2f);
    anim.SetInteger("Level", level + 1);
    yield return new WaitForSeconds(0.3f);
    level++;
    manager.maxLevel = Mathf.Max(
        level, manager.maxLevel);
    isMerge = false;
}
```

실제 레벨이 오른 뒤 갱신

level++ 다음에 maxLevel을 바꿔요.
기존 값과 현재 level 중 큰 값 저장.

Mathf.Max()

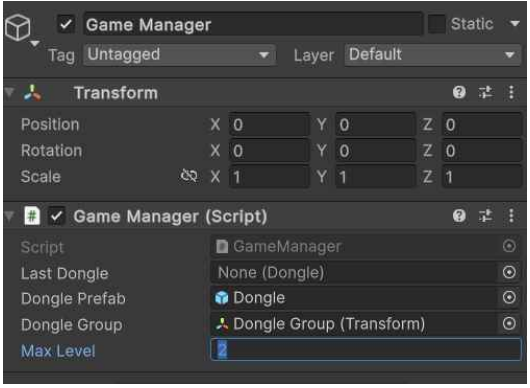
두 숫자 중 큰 값을 반환해요.
인자 순서가 바뀌어도 결과는 같아요.

최대 레벨이 내려가지 않아요

기존 maxLevel이 4이고 level이 2면
계속 4를 유지해요.
다음 Dongle 생성에 새 범위 반영.

maxLevel 설정 & 플레이 테스트

Inspector에서 GameManager의 Max Level을 2로 설정한 뒤 저장



Max Level = 2로 시작

처음에는 Random.Range(0, 2)이므로
0 또는 1만 생성돼요.

성장하면 생성 범위도 증가

3레벨 달성: Max Level 3, 0~2 생성
4레벨 달성: Max Level 4, 0~3 생성

플레이하며 Inspector의 값이 올라가는지,
2·3레벨 Dongle이 생성되는지 확인해요.

오늘 만든 합치기 로직

물리 충돌 조건과 위치로 숨길 Dongle / 성장할 Dongle 결정

HideRoutine(): 목표 위치로 이동한 뒤 비활성화
LevelUpRoutine(): Animator로 성장한 뒤 level과 maxLevel 갱신

다음 시간에는

합쳐질 때 나타나는 파티클 이펙트를 만들어요

오늘은 Dongle 합치기와 생성 레벨 갱신까지 완성!
다음 시간에 이어서 진행할게요 :)